VerbalVis: Full-Duplex Conversational Visual Analytics for Analytical Intent Revision

Anonymous Author(s)

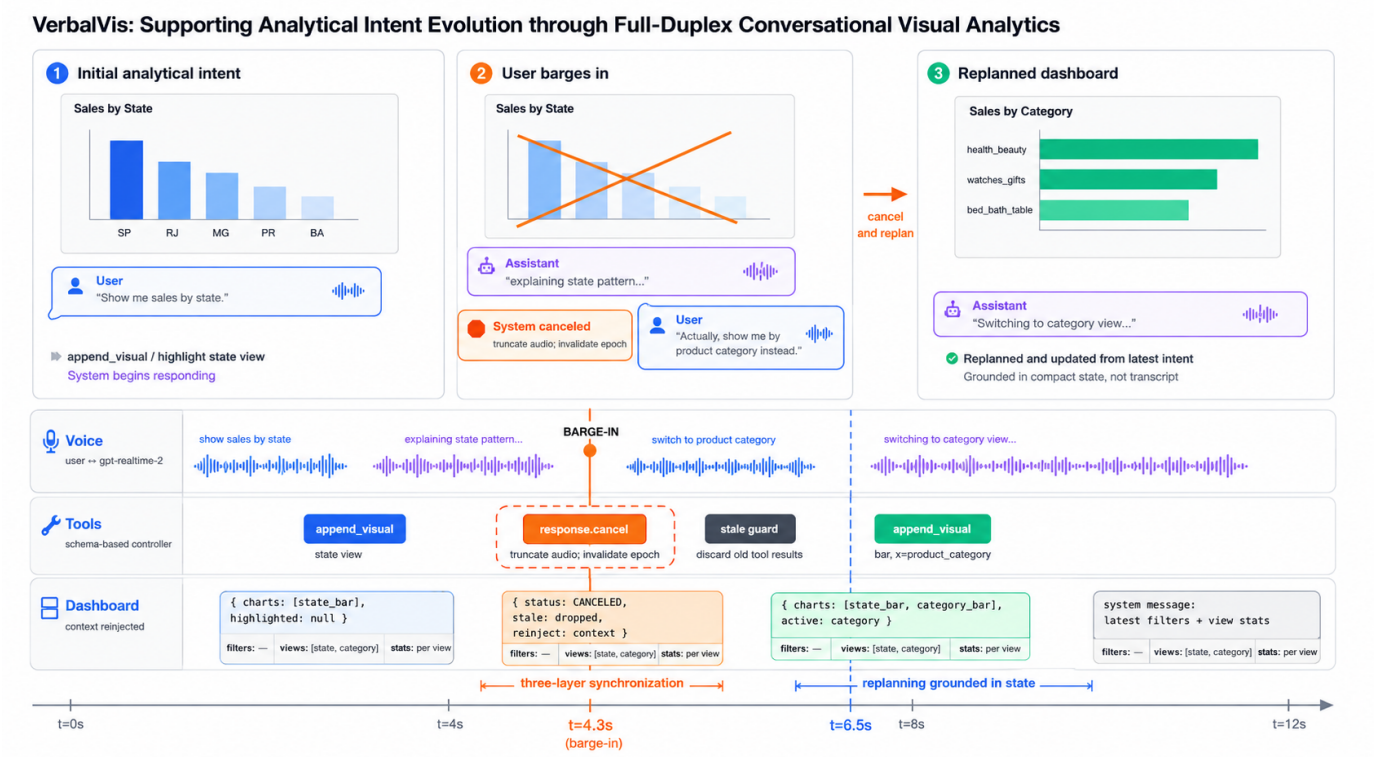

Fig. 1. Overview of a barge-in episode in VerbalVis. While the assistant is explaining a visualization, the user interrupts to redirect the exploratory analysis from sales by state to sales by product category. VerbalVis immediately cancels obsolete speech, replans visualization actions through tool execution, reinjects the updated dashboard state as contextual grounding for the language model, and synchronously updates both the dashboard and spoken response. The aligned voice, tool, and dashboard timelines illustrate the coordinated full-duplex interaction that enables continuous analytical intent evolution without explicit turn switching.

***Abstract*—Exploratory data analysis is iterative: observations may cause analysts to redirect their questions, revise provisional explanations, or change the data scope under investigation. Existing speech- and language-driven visual analytics systems generally support such revisions only after the current response has completed. Full-duplex speech interaction allows users to redirect an analysis while the assistant is still speaking, but stopping speech alone is insufficient when the superseded response may also have initiated analytical actions that modify a persistent dashboard. We present VerbalVis, a full-duplex conversational visual analytics system that coordinates spoken output, schema-grounded tool execution, and dashboard state during mid-response redirection. VerbalVis truncates obsolete speech, supersedes the active response, invalidates response-bound analytical actions, rejects stale results, and replans from the latest committed dashboard state. To describe analytical redirection, we distinguish three theory-informed, non-exhaustive, and potentially overlapping objects of revision: the analytical goal, a working hypothesis, and the analytical scope. These constructs guide design inquiry and behavioral coding rather than runtime classification. A lightweight formative inquiry informed requirements for cross-layer cancellation, state-grounded replanning, and visible execution feedback. We evaluate VerbalVis through an illustrative scenario and an exploratory within-subjects study comparing full-duplex and turn-based conversational visual analysis.**

***Index Terms*—Visual analytics, conversational interfaces, speech interaction, full-duplex dialogue, analytical intent revision, exploratory data analysis.**

Manuscript submitted to IEEE Transactions on Visualization and Computer Graphics. Source code, prompts, and study materials will be made available upon publication.

I. Introduction

EXPLORATORY data analysis (EDA) is inherently iterative. Analysts rarely follow a fully specified plan; instead, they move among observations, questions, tentative explanations, and new analytical actions as evidence emerges. Patterns and anomalies can expose limitations in the current direction and prompt analysts to reconsider what they are asking, what they believe, or which data are relevant [1]–[4]. We use *analytical intent revision* to denote an episode in

which an analyst supersedes, qualifies, or materially changes an active analytical commitment.

Natural-language interfaces and LLM-powered visual analytics systems increasingly support contextual queries, visualization generation, conversational refinement, and multistep analytical tool use [5]–[10]. These capabilities make it easier to express complex analytical requests and iteratively refine visual results. However, refinement is generally organized around completed turns: a user submits a request, the system produces an explanation or executes a sequence of analytical actions, and the user changes direction only after that response has finished. This organization is particularly restrictive for speech, where system output unfolds over time and an analyst may recognize a misdirected analysis well before the assistant has finished speaking. The interaction therefore delays the system's response to a change that has already occurred in the analyst's reasoning.

Full-duplex interaction provides a different regime by allowing the system to continue listening while speaking. An analyst can redirect an ongoing analysis through barge-in rather than waiting for a turn boundary. Yet barge-in alone does not solve the visual analytics problem. Barge-in is a temporal event, not necessarily evidence of analytical revision: a user may interrupt to correct recognition, request clarification, acknowledge the assistant, or stop an overly long response. More importantly, stopping speech does not necessarily stop the analytical consequences of the superseded response. Analytical tool actions and pending dashboard updates may continue after the user has changed direction, and late results may still modify the visible state. The assistant could therefore acknowledge the revised request while the dashboard continues evolving according to the obsolete one. Full-duplex conversational visual analytics consequently introduces a cross-layer consistency problem spanning spoken output, analytical execution, and persistent dashboard state.

We present **VerbalVis**, a full-duplex conversational visual analytics system designed to keep these layers aligned during mid-response redirection. VerbalVis combines real-time speech interaction, schema-grounded analytical tools, and a persistent dashboard through a visual-analytics-specific orchestration layer. When a new request supersedes an active response, the system truncates obsolete speech and advances a response-bound execution epoch. Tool actions remain associated with the response that produced them; actions and results from earlier epochs are treated as stale and cannot commit changes to the dashboard. The revised request is then interpreted against the latest committed dashboard state, including active filters, highlights, and visualizations, so that replanning begins from what the analyst currently sees rather than from speculative work associated with the cancelled response. VerbalVis does not introduce a new speech model or interruption detector. Its contribution is the response-supersession mechanism that coordinates speech cancellation, analytical validity, and dashboard-grounded replanning.

Drawing on research in sensemaking and exploratory analysis, we characterize analytical revisions through three potentially overlapping objects of change: the analytical goal, a working hypothesis, and the analytical scope [2], [4], [11], [12]. These constructs provide a vocabulary for describing how an active investigation is redirected, but they do not imply that every interruption constitutes a revision or that every revision belongs to only one category. They serve as theory-informed sensitizing concepts and post-hoc coding dimensions rather than an exhaustive taxonomy or runtime intent classes. A lightweight formative inquiry refined their operational boundaries and informed requirements for cancellation, state preservation, visible feedback, and recoverable dashboard interaction.

We evaluate VerbalVis through an illustrative usage scenario and an exploratory within-subjects mixed-methods study with eight participants. The study compares a *Full-Duplex* condition, in which participants may redirect the assistant while it is speaking, with a matched *Turn-Based* condition, in which they must wait for the current response to finish. We examine interaction responsiveness, perceived control, state consistency, workload, exploratory behavior, and analytical outcomes, with particular attention to when participants revise an ongoing analysis and what they expect the system to cancel, preserve, and update.

Contributions.

1) A design framing for overlapping user speech in full-duplex conversational visual analytics. We distinguish non-interruptive backchannels from floor-taking barge-ins and characterize analytical revisions through changes to analytical goals, working hypotheses, and analytical scope.
2) **VerbalVis**, a full-duplex conversational visual analytics system that coordinates assistant speech, analytical tool execution, and persistent dashboard state when an active analytical response is superseded. It supports speech truncation, response-bound tool execution, stale-result rejection, and replanning from the latest committed dashboard state.
3) An exploratory mixed-methods comparison of full-duplex and turn-based speech interaction during open-ended visual analysis, examining responsiveness, perceived control, interaction flow, state understanding, workload, exploratory behavior, and analytical outcomes.

II. Related Work

VerbalVis connects research on natural-language interaction for visualization, LLM-powered visual analytics, full-duplex spoken dialogue, and analytical sensemaking. We review how these areas support conversational refinement, analytical steering, interruption, and changes to an ongoing investigation.

A. Natural-Language Interaction for Visualization

Natural-language interfaces for visualization translate utterances into data attributes, analytical tasks, and visual specifications. DataTone exposes alternative interpretations through interactive ambiguity widgets [5], while Eviza grounds language in both the dataset and the active visualization to support contextual references and follow-up questions [6]. NL4DV provides a reusable pipeline for extracting attributes and

analytical tasks and generating candidate visualizations [7]. These systems establish language grounding, ambiguity management, and contextual interpretation as central requirements for conversational visual analysis.

Multimodal systems extend this interaction by combining language with visible references and direct manipulation. Orko integrates speech and touch for network exploration [13]; InChorus characterizes operations expressed through speech, pen, and touch [14]; and DataBreeze interweaves speech with direct manipulation over flexible unit visualizations [15]. Snowy and conversational extensions of NL4DV further support follow-up utterances and modification of existing views [16], [17]. However, these systems generally organize refinement around completed user and system turns. VerbalVis instead considers redirection while speech and associated analytical actions are still unfolding.

B. LLM-Powered and Agentic Visual Analytics

Large language models extend conversational visualization beyond mapping one utterance to one chart. LIDA decomposes visualization generation into data summarization, goal generation, chart creation, and evaluation [8]. Data Formulator combines natural-language concepts with directly specified visual encodings, while Data Formulator 2 supports iterative authoring and navigation through alternative data and visualization histories [9], [18]. These systems broaden the role of language from chart specification to data transformation and iterative visual design.

Agentic systems further delegate analytical planning and execution to LLM-based components. InsightPilot and LightVA translate higher-level analytical goals into sequences of tasks, visualizations, and findings [10], [19]. WaitGPT externalizes intermediate analysis operations for inspection and modification [20], while InsightLens links conversation, code, visualizations, and findings across an extended analysis [21]. Such systems make analytical processes more steerable and inspectable. Nevertheless, steering is usually performed through subsequent prompts, graphical controls, or explicit checkpoints. Existing work rarely specifies whether an unfinished action generated by an earlier response remains valid after the user has redirected the analysis.

C. Full-Duplex Dialogue and Interruption

Streaming speech systems incrementally process or generate audio, whereas full-duplex systems continue listening while speaking. They must distinguish background speech, backchannels, cooperative overlap, and attempts to take the conversational floor [22]. Barge-in has traditionally referred to detecting user speech during system output and stopping or modifying the active response [23]. Recent systems including Duplex Conversation, SyncLLM, Moshi, and SALMONN-omni investigate concurrent speech streams, low-latency turn-taking, and context-sensitive interruption [24]–[27].

Interruption timing, however, does not determine analytical meaning. A user may interrupt to acknowledge the assistant, repair recognition, request clarification, or revise the investigation. Conversely, an analyst may revise the analysis after the assistant has finished speaking. Moreover, stopping audio does not automatically cancel a database query, visualization tool call, or pending dashboard update. VerbalVis therefore treats barge-in as a temporal signal while separately managing the validity of response-dependent analytical work and persistent visual state.

D. Sensemaking and Analytical Revision

Sensemaking research characterizes analysis as an iterative process in which questions, explanations, and data focus evolve as evidence is encountered [1]–[3]. For *Analytical Goal Shift*, information-seeking studies describe evolving queries and the refinement of broad goals into focused analytical subtasks [4], [12], [28]. For *Working-Hypothesis Revision*, Data–Frame Theory and scientific reasoning show that provisional explanations may be questioned or replaced when observations conflict with prior expectations [11], [29], [30]. For *Analytical Scope Refinement*, information-foraging and visualization task frameworks distinguish analytical purpose from the data target and its progressive restriction by population, variable, time, geography, category, or granularity [31]–[33].

These traditions provide theoretical support for the three forms, but do not establish a universal, exhaustive, or mutually exclusive taxonomy. We therefore treat them as theory-informed sensitizing concepts. Section III presents participants' utterances as empirical instances and specifies the coding boundaries adopted in this work; it does not serve as the theoretical evidence that these forms of revision exist.

Across these areas, prior work supports contextual language interaction, multistep analytical execution, spoken interruption, and evolving analytical histories. VerbalVis connects these capabilities by coordinating the validity of spoken output, analytical actions, and persistent dashboard state when an unfolding response is superseded.

III. Formative Inquiry and Design Requirements

We conducted a formative inquiry with four participants (F01–F04) to understand how analysts revise an ongoing investigation during conversational visual analysis and what system behavior such revisions require. The inquiry was design-oriented rather than confirmatory: it aimed to refine operational definitions, identify compound and boundary cases, and inform the design of VerbalVis, rather than establish an exhaustive taxonomy of analytical behavior.

A. Theory-Informed Revision Constructs

Exploratory analysis is iterative: observations can redirect questions, reshape provisional explanations, and change which data are considered relevant [1], [2], [4]. Research on sensemaking and scientific reasoning further shows that analysts construct and revise explanatory frames as they encounter new evidence [11], [29], [34]. Information-seeking and visualization-task research similarly distinguishes changes in analytical purpose from changes in the data target and means of analysis [12], [28], [32].

Based on these traditions, we specified three theory-informed sensitizing concepts before analyzing the formative sessions:

Analytical Goal Shift. The analyst materially redirects the primary analytical question or desired knowledge outcome.

Working-Hypothesis Revision. The analyst rejects, replaces, or qualifies an active provisional explanation, interpretation, or decision-relevant proposition.

Analytical Scope Refinement. The analyst adds, removes, or changes constraints over the relevant population, time period, geography, variables, categories, granularity, or data subset while generally preserving the higher-level analytical objective.

These constructs describe different objects of analytical change rather than mutually exclusive event classes. A single utterance may therefore receive multiple labels. They are also non-exhaustive: hypothesis formation, method changes, representation changes, evidence requests, and conversational repair were retained as separate boundary codes.

B. Procedure and Analysis

Participants used an early VerbalVis prototype to explore the Olist Brazilian e-commerce dataset through speech. The initial dashboard presented five coordinated views: monthly order volume, category-level sales, state-level order distribution, review-score distribution, and delivery-time statistics. The exploration was open-ended; participants were free to pursue questions, comparisons, explanations, and data subsets that they considered relevant. They were not introduced to the three revision constructs and were not required to interrupt the assistant or follow a prescribed analytical path.

We recorded timestamped user and assistant utterances together with available interaction and dashboard events. After the exploration, participants completed a post-use questionnaire concerning when and why they changed direction, what state should be preserved or abandoned, their perceived control, and the dashboard operations they considered important.

Our analysis followed a hybrid deductive–inductive process. We first used an LLM-assisted pass to retrieve candidate episodes from the logs. Each candidate included the exact utterance, its preceding analytical commitment, proposed labels, timing, and a rationale. We then manually reviewed every candidate against the surrounding dialogue and dashboard context.

An episode was retained as an *analytical revision* only when an active analytical commitment was identifiable before the utterance and the utterance materially changed, qualified, or superseded it. Initial formation of a question or explanation was distinguished from revision of an existing one. Likewise, a change in chart, metric, or calculation procedure was not treated as a goal, hypothesis, or scope revision unless the corresponding analytical commitment also changed.

The final coding retained 27 analytical revision episodes. Using a multi-label scheme, 12 episodes involved an Analytical Goal Shift, five involved a Working-Hypothesis Revision, and 18 involved an Analytical Scope Refinement. Eight episodes involved more than one construct. Because the categories were not mutually exclusive, their counts do not sum to the number of episodes.

Interaction timing was coded separately from revision semantics. This allowed us to distinguish *what* changed in the investigation from *when* the change was expressed relative to system speech, analytical execution, and dashboard updates.

C. Formative Findings

1) Participants revised different analytical commitments: All four participants reported changing analytical direction during the exploration, and three reported doing so at least twice. All four reported changes to the data scope, three reported changes to their interpretations or conjectures, and two reported changes to the primary analytical question. These responses were consistent with the manually coded logs.

Goal shifts changed the knowledge outcome being pursued. Participants moved, for example, from monthly order trends to low-rating customer experience, from delivery performance to payment behavior, and from general comparison to the active search for anomalous products.

Scope refinements preserved the broader objective while changing the data target. They included geographic grouping, rating thresholds, time intervals, product-category substitution, and changes from monthly to weekly granularity. Scope changes included narrowing, broadening, substitution, and regrouping; we therefore use *Scope Refinement* rather than the narrower term *Scope Narrowing*.

Working-hypothesis revisions occurred when participants questioned or qualified an active explanation or decision criterion. They were distinguished from hypothesis formation, in which an explanation was introduced for the first time, and from method revision, in which the analyst changed a metric or calculation without necessarily changing an explanatory proposition.

2) Revision constructs overlapped within individual utterances: Eight episodes changed more than one analytical object. For example, one participant requested:

> "Apply the same analysis to the third- through fifth-ranked states; I want to find products with relatively low ratings."

This request simultaneously restricted the geographic scope and redirected the analytical purpose from general comparison to anomaly seeking. We therefore coded it as both Analytical Scope Refinement and Analytical Goal Shift.

Such episodes indicate that goal, hypothesis, and scope are related but distinguishable dimensions of analytical commitment. They should not be implemented as mutually exclusive runtime intent classes. Instead, the system must determine which prior commitments remain valid and compose the actions required by the latest request.

3) Analytical revision was distinct from repair and method change: Speech overlap did not necessarily indicate analytical revision. Participants also spoke during system output to correct recognition errors, ask immediate follow-up questions, stop an overlong explanation, or provide conversational acknowledgements. For example, "I mean by state—state"

TABLE I
REPRESENTATIVE EPISODES FOR THE THREE ANALYTICAL REVISION CONSTRUCTS. UTTERANCES ARE TRANSLATED FROM CHINESE.

Construct	Prior commitment	User utterance	Interpretation
Analytical Goal Shift	Explore monthly order trends.	"Analyze customer experience—the low-rated orders, including their time, category, and delivery conditions."	The desired outcome changes from describing order volume to investigating low-rating customer experience and its potential contributing factors.
Working-Hypothesis Revision	Prioritize Bahia because it has the longest delivery time among high-volume states.	"BA only has the longest delivery time. Does this take sales volume into account?"	The user qualifies the active proposition by questioning whether delivery time alone is sufficient and introduces sales volume as an additional decision criterion.
Analytical Scope Refinement	Investigate customer experience using the full set of orders.	"Now filter the global review scores to one and two."	The higher-level objective remains unchanged, while the relevant population is restricted to low-rating orders for subsequent analysis.

restored a likely misrecognized request rather than expressing a new analytical direction.

Changes in chart form or metric were similarly context-dependent. Combining delivery days and order volume into a ratio changed the analytical operationalization, but did not necessarily revise an explanation or data scope. These boundary cases required interpretation of the preceding commitment rather than classification from the latest utterance alone.

4) Participants expected selective cancellation and preservation: Three participants indicated that the assistant should stop its current speech when a new direction was expressed. Participants also expected useful committed state to be preserved: three selected still-relevant visualizations, two selected previously confirmed findings, and two selected prior conversational context.

By contrast, three participants indicated that unfinished explanations and unfinished analyses from the old direction should be abandoned, two selected pending visual updates, and all four selected details unrelated to the revised question. These responses suggest that redirection requires selective supersession rather than either preserving or resetting the entire analytical state.

Participants' main concerns included speech-recognition errors, misinterpretation of analytical intent, overly long responses, and an obsolete direction continuing to affect the dashboard. All four agreed that they could change direction when needed and understood which request the dashboard represented. Three reported an overall sense of control, while one was neutral. Opinions about response length were divided, reinforcing the need for concise and interruptible responses.

D. Design Requirements

The formative findings informed four design requirements.

a) DR1: Ground interpretation in the current analytical and visual state.: The system should interpret each new utterance relative to the ongoing conversation and the current state of the analysis, including the active question, relevant working explanations, filters, highlights, and visualizations. This grounding is necessary to determine whether the utterance continues the current analysis, qualifies part of it, or redirects it toward a different analytical direction.

b) DR2: Support compound revisions through composable analytical actions.: A single utterance may change more than one aspect of the analysis. For example, a user may simultaneously redirect the analytical goal and restrict the relevant data scope. The system should therefore support composable, schema-grounded action sequences rather than map each utterance to a single operation or an exclusive revision category.

c) DR3: Avoid treating every interruption or conversational repair as analytical revision.: Speech overlap may indicate an analytical revision, but it may also reflect an acknowledgement, an ASR correction, a clarification request, or an attempt to shorten an explanation. Similarly, a request to change a chart or metric does not necessarily change the underlying analytical direction. The system should interpret the utterance in context and preserve valid analytical state when the user is repairing communication or modifying the method rather than replacing the ongoing investigation.

d) DR4: Coordinate redirection across speech, analytical execution, and visual state.: When a new request redirects an active analysis, stopping audio alone is insufficient. Pending or running response-dependent actions should be cancelled where possible, and results associated with an obsolete response should be prevented from modifying the dashboard. Subsequent planning should begin from the latest committed dashboard state while preserving filters, visualizations, findings, and dialogue context that remain relevant to the revised request.

These requirements directly informed the design of VerbalVis. Compact conversation and dashboard-state grounding supports DR1; schema-grounded, composable visualization tools support DR2; contextual interpretation of overlapping speech supports DR3; and response-bound execution epochs, stale-result rejection, and replanning from the latest committed dashboard state support DR4.

E. Scope of Evidence

This formative inquiry provides design-oriented evidence rather than validation of a universal taxonomy. Four participants cannot establish theoretical saturation, estimate population-level prevalence, or demonstrate that the three constructs cover every form of analytical change.

We therefore treat Analytical Goal Shift, Working-Hypothesis Revision, and Analytical Scope Refinement as theory-informed, non-exhaustive, and potentially overlapping analytical lenses. The 27 verified episodes characterize the present formative corpus and inform the design of VerbalVis. The subsequent user study evaluates the resulting full-duplex interaction and orchestration mechanisms rather than testing the three constructs as a universal classification.

IV. VerbalVis Design

VerbalVis is designed around a continuous exploratory loop in which visual evidence and spoken interaction repeatedly inform one another. An analyst first observes the dashboard, expresses an analytical request through speech, and receives a spoken response and, when necessary, one or more tool-supported updates to the dashboard. The resulting visual state may reveal a trend, difference, outlier, or counterexample that motivates the analyst to continue, qualify, or redirect the analysis. We therefore treat interaction with VerbalVis not as a sequence of isolated natural-language-to-chart commands, but as an ongoing cycle:

$$\begin{aligned}\text{Visual Observation} &\rightarrow \text{Spoken Analytical Request}\\ &\rightarrow \text{Tool-Supported Analysis}\\ &\rightarrow \text{Dashboard Update}\\ &\rightarrow \text{New Visual Observation.}\end{aligned} \tag{1}$$

Full-duplex speech changes the design requirements of this cycle. A user may form a new question while the assistant is still explaining the previous result and may express the new request immediately rather than waiting for a turn boundary. Merely stopping the assistant's audio is insufficient in this setting: analytical operations associated with the superseded response may already have been requested, may still be executing, or may return after the user has redirected the analysis. VerbalVis consequently coordinates spoken output, analytical actions, and dashboard state. The following design describes how the four requirements derived from our formative inquiry are reflected in the interaction model.

A. Interaction Model

During an uninterrupted interaction, the user begins with an overview dashboard and poses a question such as, "Show me the low-rated orders." The assistant may briefly acknowledge the request, invoke a filtering operation, and summarize the resulting state after the dashboard has been updated. The user can then inspect the revised distributions and rankings before posing a follow-up question. Speech provides a rapid means of expressing analytical direction, whereas the persistent dashboard preserves the evidence required for comparison and subsequent reasoning.

In the full-duplex condition, VerbalVis continues receiving user audio while the assistant is producing speech. The user therefore does not need to issue a separate stop command before redirecting the analysis. For example, while the assistant is explaining differences in order volume across states, the user may say, "Wait, I am more interested in the low-review orders." From the user's perspective, the assistant yields the audio channel, accepts the new utterance, and continues the interaction from the revised request.

A redirection does not imply that the complete analytical workspace should be reset. Results that have already become part of the visible analysis may remain relevant to the new request. For example, after filtering the data to low-review orders, the user may redirect an unfinished category analysis toward delivery time. The low-review filter should normally remain active, while an uncompleted category-specific action associated with the superseded response should not determine the next dashboard state. This selective preservation allows an analyst to revise one aspect of an investigation without repeatedly reconstructing all previously established constraints.

We use *Analytical Goal Shift*, *Working-Hypothesis Revision*, and *Analytical Scope Refinement* as non-exhaustive and potentially overlapping dimensions for describing such changes in the formative inquiry and user-study analysis. They characterize what an analyst revised; they are not mutually exclusive intent labels, a complete taxonomy of conversational behavior, or three runtime states used to dispatch system actions. VerbalVis does not require an explicit classifier that assigns every utterance to one of these dimensions.

B. Context-Grounded Interpretation

Natural-language requests in an ongoing analysis are frequently elliptical. Utterances such as "Only keep SP," "Compare that with delivery time," or "Highlight the second one" depend on entities and constraints that are already visible or were recently discussed. Interpreting these utterances in isolation would require the user to restate the full analytical query on every turn. To support concise follow-up requests, VerbalVis interprets a new utterance relative to the current conversational and visual state (**DR1**).

Conversational context provides the recent analytical topic, variables or groups mentioned by the user, the aspect currently being explained by the assistant, and the operations already completed during the interaction. VerbalVis does not represent the user's goal or working hypothesis as a formal symbolic object. Instead, the dialogue history helps the model resolve references and continue the current line of inquiry. This distinction avoids conflating the analytical dimensions used in our research coding with the runtime representation used by the system.

Visual grounding complements the dialogue context. The relevant state includes the active data filters, the currently highlighted view, the identifiers and titles of available views, their chart types, and compact summaries of the data displayed in them. For instance, interpreting "highlight the second one" requires an ordered inventory of the available views, while "continue with the low scores" requires knowing whether a

review-score constraint has already been applied. The dashboard therefore functions as part of the conversational context rather than as a passive output surface.

The design principle is that planning should begin from the latest state that has been accepted as part of the current analysis, not from a speculative result that has not yet been presented. VerbalVis supplies a compact dashboard summary instead of repeatedly sending the full dataset, complete visualization specifications, or an unbounded interaction history. This representation keeps the model grounded in the state most relevant to the next request while limiting unnecessary context.

C. Composable Analytical Actions

An analytical utterance may require several coordinated actions (**DR2**). Consider the request, "Only show low-review orders from SP, and plot delivery time against review score." This utterance combines at least two scope constraints with the creation of a new relationship view. Treating the complete request as a single indivisible intent, or assuming that each utterance maps to exactly one tool, would either omit part of the request or force the user to decompose it manually.

VerbalVis therefore exposes a set of composable analytical actions. Filtering actions modify the data population under analysis. Highlighting actions direct visual attention without necessarily changing the underlying rows. View creation actions add a new representation to the workspace so that the user can inspect a relationship not shown in the initial dashboard. Additional actions allow the user to remove filters, adjust the operational threshold used to denote low-review orders, and delete no-longer-needed workspace views.

The tools are composable because analytical revisions and visualization operations are not in one-to-one correspondence. A scope refinement may require two filters and an appended chart; a goal shift may reuse the current scope but create a different view; and a working-hypothesis revision may combine a new comparison with a change in visual attention. The system may therefore issue multiple structured calls for one user utterance, with their effects accumulated in the shared analytical workspace.

Schema grounding constrains these actions to operations supported by the actual dataset and interface. The model selects among registered functions and supplies structured arguments rather than directly generating arbitrary SQL or unconstrained visualization code. This makes the resulting actions easier to validate, record, execute, and associate with the response that requested them. It also reduces the likelihood that a conversational model invents a field, transformation, or visual operation that the dashboard cannot execute.

D. Interpreting Overlapping Speech

VerbalVis distinguishes the temporal occurrence of overlapping speech from the semantic phenomenon of analytical revision (**DR3**). A *barge-in* occurs when the user begins speaking while the assistant is still producing output. An *analytical revision* occurs when the content of a user utterance changes the current analytical direction. The two are related because full-duplex interaction gives users an immediate opportunity to revise an analysis, but they are not equivalent.

Overlapping speech may serve several functions. A user may produce a backchannel such as "okay," correct an ASR error, request clarification, ask the assistant to stop, or express a new analytical request. Background speech may also be detected as user speech. Consequently, the occurrence of speech overlap alone is not evidence that the user has changed the analytical goal, working hypothesis, or scope.

At the speech level, VerbalVis adopts a conservative yielding policy. When user speech is detected during assistant output, the assistant stops occupying the audio channel so that its speech does not mask the user's utterance. This decision concerns conversational floor management. It does not automatically erase the visible dashboard, undo previously accepted actions, or determine the semantic function of the new utterance.

After the utterance is available, the conversational model interprets it using the recent dialogue and dashboard context. A correction may repair a misrecognized value, a clarification may be answered without changing the analysis, and an analytical redirection may trigger a different set of tools. The current prototype does not contain an independent semantic classifier for backchannels, repairs, clarifications, and analytical revisions. Instead, these utterances pass through the same context-grounded conversational pipeline.

This design prioritizes immediate floor yielding over preserving interrupted assistant speech. Thus, even a short acknowledgement may stop the active response, and the system does not automatically resume the exact remainder of that response. We treat this behavior as a limitation of the present prototype rather than evidence that every detected overlap constitutes an analytical revision.

E. Coordinated Redirection and State Preservation

Redirection creates a coordination problem across three processes: the assistant's spoken response, analytical tool execution, and visual-state updates (**DR4**). Suppose that the assistant is explaining category revenue and has requested a category-related visualization. Before the operation finishes, the user redirects the analysis toward low-review orders. If the old result is later applied to the dashboard without checking which response produced it, the visible state may no longer correspond to the user's latest request.

Stopping audio addresses only the first process. An already generated tool call may be queued, a database query may be executing, or a dashboard update may be awaiting delivery. VerbalVis therefore associates analytical work with the conversational response under which it was initiated and attempts to invalidate work owned by a superseded response.

Conceptually, the design distinguishes *established analytical state* from *pending work*. Established state includes filters, views, and highlights that have already become part of the user's workspace. Pending work includes tool calls and results associated with an active response that have not yet been accepted for presentation. This distinction supports selective preservation: established state is retained unless the new

utterance explicitly removes or replaces it, whereas pending work from a superseded response should not be used to communicate or visualize a result.

For example, if a low-review filter has already been applied and the user interrupts an explanation about product categories to ask about delivery time, the filter remains useful context. In contrast, a category view that was requested by the interrupted response but has not yet been presented should not become the basis of the next spoken answer. Replanning should instead use the visible workspace and the newly completed utterance.

The general coordination policy is therefore to cancel obsolete work where the underlying component supports cancellation and to mark it as logically invalid otherwise. Logical invalidation means that a computation may still finish, but its result should not be relayed as a valid answer for the current request. This policy is intended to reduce stale updates; it does not imply that every underlying computation can be transactionally rolled back.

F. Visualization as a Persistent Analytical Workspace

VerbalVis combines speech with a persistent visual workspace because spoken answers alone are transient and sequential. They are poorly suited for comparing many values, inspecting distributions, or revisiting evidence mentioned several conversational turns earlier. A dashboard allows the user to inspect patterns that the assistant did not explicitly verbalize and to verify whether a spoken interpretation is supported by the displayed data.

The initial dashboard provides four complementary entry points into the Olist data: monthly order volume, review-score distribution, order volume by customer state, and the top product categories by revenue. Together, these views expose temporal, evaluative, geographic, and categorical perspectives. They are intended as starting points rather than a fixed analytical report.

The dynamic workspace extends this overview with charts created during the conversation. A user can request a relationship involving delivery time, review score, category, state, or another supported variable, and the appended view remains available for subsequent reference. Filters are reflected across the relevant views, while highlighting can direct attention to a particular view without changing the underlying population. Users can also remove filters or delete workspace views as the investigation evolves.

The roles of the modalities are thus complementary: visualization supports observation, comparison, and verification; speech supports rapid expression of questions and redirections; structured tools translate language into executable analytical operations; and coordination mechanisms attempt to keep spoken and visual outputs aligned with the current request.

G. Interaction Feedback

Full-duplex interaction can create uncertainty about whether the system is listening, whether a tool is executing, and whether the visible dashboard has already incorporated a request. VerbalVis therefore exposes lightweight feedback through the microphone state, connection state, final transcripts, active-filter badges, and chips representing recent tool calls.

These elements help users identify the current data scope and inspect which operations have recently affected the dashboard. The transcript also provides a persistent record of recognized speech, which is particularly important when an analytical action depends on a state name, threshold, or other value that may have been misrecognized.

The present prototype does not expose a complete visualization of every internal phase. In particular, it does not visibly distinguish all speaking, reasoning, cancellation, and dashboard-rendering states. We therefore describe the feedback layer as lightweight rather than as a comprehensive execution monitor. Improving interruption acknowledgement and state-transition feedback is an important direction for subsequent iterations.

H. Design Summary

Table II summarizes how the design responds to the four requirements. Together, these decisions make the dashboard an evolving analytical workspace rather than an output generated independently for each turn. Context grounding links new requests to the visible analysis, composable actions support compound utterances, conservative overlap handling separates floor yielding from semantic interpretation, and cross-stream coordination attempts to prevent superseded results from being presented as current.

V. System Implementation

VerbalVis is implemented as a browser-based, single-user research prototype comprising a Vue frontend, a FastAPI backend, a realtime speech model, schema-grounded analytical tools, an in-memory DuckDB database, and a persistent Vega-Lite dashboard. The implementation has four primary coordination objectives: streaming user and assistant audio with low delay, binding tool work to the response that initiated it, rejecting results from superseded responses before they are relayed, and providing subsequent model responses with a compact representation of the current dashboard.

The active implementation uses Qwen 3.5 Omni Plus Realtime through the DashScope WebSocket API. Although the repository contains a separate reference implementation for an OpenAI realtime model, that path is not connected to the active server routes and is not used in the system reported here.

A. System Architecture

The frontend is implemented with Vue 3 and Vite. Pinia stores the client-side dashboard state, while Vega and Vega-Lite render the visualizations. The frontend is responsible for microphone capture, assistant-audio playback, transcript presentation, interaction feedback, and dashboard rendering. Audio and application messages are exchanged with the backend through a persistent WebSocket connection.

The backend is implemented with FastAPI and exposes a primary `/ws` endpoint. For each connection, it creates a

TABLE II
MAPPING FROM DESIGN REQUIREMENTS TO VERBALVIS DESIGN RESPONSES.

Req.	Design response	User-facing capability	System responsibility
DR1	Context-grounded interpretation	Follow-up requests can refer to active filters, visible views, and recently discussed variables.	Provide compact dialogue and dashboard state when interpreting new speech.
DR2	Composable analytical actions	One utterance can filter, redirect attention, and create a new visualization.	Translate the request into one or more validated, schema-grounded tool calls.
DR3	Contextual handling of overlap	The assistant yields when the user speaks, without assuming that every overlap is an analytical revision.	Separate speech-onset handling from interpretation of the completed utterance.
DR4	Coordinated redirection	Superseded speech and pending analytical work are prevented, where possible, from determining subsequent output.	Associate tools with responses, cancel work when possible, and logically reject stale results.

realtime-session manager that relays audio and control events between the browser and the Qwen DashScope WebSocket. The session manager also receives model function calls, dispatches them to the analytical tool layer, records response and tool state, and forwards accepted results to the frontend.

The data layer uses an in-memory DuckDB instance populated from the Olist Brazilian e-commerce data. It maintains an order-level table containing delivered orders and an item-level table containing order-item records. The order-level table supports temporal, review, state, and delivery analyses, whereas the item-level table supports category and revenue aggregation. Database queries are generated by the backend tool handlers after model arguments have passed field and operation validation.

The backend currently stores dashboard data in process-level mutable structures. Consequently, the deployed prototype assumes one active analytical session rather than concurrent, isolated multi-user sessions. This architecture is sufficient for the controlled user study but would need session-scoped state containers for multi-user deployment.

B. Full-Duplex Audio Pipeline

The browser captures microphone input using an `AudioWorklet`. Input is converted to 16-kHz mono PCM16 and divided into approximately 100-ms chunks. Each chunk is base64 encoded and sent through the application WebSocket to the FastAPI backend, which relays it to the Qwen realtime connection. Audio is uploaded continuously in the full-duplex mode; although a client-side VAD implementation remains in the codebase, the active configuration does not gate silent chunks.

Qwen's server-side VAD detects the beginning and end of user speech. The active configuration uses a speech threshold of 0.5 and an 800-ms silence interval. When the end of a user utterance is detected, the provider automatically initiates a model response. The response may contain streaming audio, incremental transcripts, and structured function calls.

Assistant audio is returned as 24-kHz PCM16 deltas. The frontend decodes the chunks and schedules them through an `AudioContext`. A `nextPlayTime` anchor is used to place successive chunks on a continuous playback timeline, allowing speech to begin before the full model response has been generated.

When the server VAD emits a user-speech-start event during assistant output, the backend invokes its response-invalidation procedure. It increments the current turn epoch, records the active response as invalidated, requests cancellation of tracked tool tasks, sends a `response.cancel` message to Qwen, and forwards a speech-start notification to the browser. Upon receiving that notification, the frontend stops assistant playback by closing the active `AudioContext`. Thus, audio termination occurs at the client, while response and tool invalidation are initiated at the backend.

The active Qwen API does not support truncating the assistant conversation item at the exact amount of audio heard by the user. VerbalVis therefore cannot remove the unheard suffix from the provider-side conversation using an operation equivalent to `conversation.item.truncate`. This differs from the client-side playback stop: the user no longer hears the audio, but the provider may retain generated content in its internal conversational context. The current implementation also does not reliably log the exact frontend playback-stop time, which limits measurement of wasted speech.

The turn-based experimental condition uses the same model, prompt, tools, database, and dashboard. It is configured at server startup using the `VERBALVIS_BARGE_IN_ENABLED` environment variable. When barge-in is disabled, provider-side response interruption is disabled and the frontend microphone control is unavailable while the assistant is speaking. The mode is therefore changed between experimental sessions rather than switched by an end user during a session.

C. Prompting and Dashboard-State Grounding

The Qwen system prompt is assembled from nine instruction sections describing the assistant's analytical role, the available data fields, tool-selection rules, conversational behavior, response style, and error handling. The instructions encourage concise spoken feedback and require the model to use registered tools for operations that modify the dashboard. They also instruct the model to prioritize the most recent completed user utterance when the interaction has been redirected.

VerbalVis partially decouples spoken acknowledgement from analytical execution. The assistant can briefly acknowledge a request while the model produces function calls for the corresponding dashboard actions. This avoids requiring a long verbal explanation to finish before any visual operation can begin. The spoken answer and tool results are nevertheless generated within the same realtime conversational session rather than by independent agents.

The backend constructs a compact dashboard-context string through `context_text()`. The representation includes active filters, the configured low-score threshold, the current number of matching rows, the highlighted view, view identifiers, titles and chart types, and per-view statistics. These fields allow the model to resolve requests that refer to the currently visible analysis without transmitting the full underlying tables or complete Vega-Lite specifications.

The dashboard context is included in the session instructions when the Qwen connection is initialized. After a successful tool call, an updated context is also included with the function-call output returned to the model. The active provider path does not support inserting an arbitrary new system-message item between tool calls. Consequently, dashboard context is refreshed at initialization and after tool execution, but it is not independently pushed after every intermediate client-side state change. This is a known limitation of the current grounding mechanism.

D. Schema-Grounded Analytical Tools

The active implementation registers six analytical functions with the realtime model. Each function is described by a parameter schema and is implemented by a corresponding server-side handler. Model-generated arguments first pass through `normalize_tool_arguments()`, which repairs or infers selected parameters from the recognized utterance, and are then validated against server-side field and operation constraints before execution.

Table III summarizes the registered tools. The tools operate on the shared dashboard state rather than returning standalone chart code. Filtering and threshold operations alter the data used to refresh views; highlighting changes the current visual focus; and append and delete operations modify the dynamic workspace.

The normalization layer supports common variations in spoken requests, including category names, state references, sorting directions, Top-N requests, low-review thresholds, and requests whose visual form is implied rather than explicitly named. The server still performs the final validation. In particular, `append_visual` uses a multi-stage validation chain to check the requested chart type, fields, aggregation requirements, and compatibility of the requested encodings. The current implementation performs 14 checks before adding an accepted dynamic visualization.

One model response may contain multiple function calls. The backend records the number of pending calls associated with a response and delays creation of a subsequent model response until the calls have been finalized. A shared `_tool_state_lock` serializes state-changing tools, so multiple calls from one utterance are coordinated but are not executed concurrently. The implementation does not currently construct an explicit dependency graph between calls; their shared state is determined by the serialized execution order.

E. Data and Visualization State

At session initialization, the backend resets the shared analytical state and constructs four base views:

1) `view-trend`: monthly order counts;
2) `view-review`: the distribution of review scores;
3) `view-map`: order counts by customer state; and
4) `view-category`: the top 15 product categories by revenue.

Despite the identifier `view-map`, the current implementation presents the state distribution through a categorical chart rather than a geographic map.

The server-side state consists primarily of `active_filters`, `views`, `highlighted_view`, `workspace_counter`, and `low_score_threshold`. The `views` collection contains both the initial views and dynamically appended workspace views. `workspace_counter` generates identifiers of the form `workspace{N}` for appended views. The backend is the authoritative source for the data and specifications contained in these views.

After a state-changing tool succeeds, the backend recomputes the affected visual data and sends a `views_update` message. The Pinia store's `updateViews()` operation replaces the frontend view collection with the received state, after which the Vega-Lite components rerender. This whole-state update protocol simplifies synchronization for a single-user prototype.

The current implementation does not attach a monotonically increasing dashboard version to these updates. The frontend therefore performs no version, sequence, or epoch comparison before applying a `views_update`. If an obsolete update were to reach the browser, it would be accepted unconditionally. VerbalVis currently relies on backend-side stale-result checks to reduce this risk; a client-side version check would provide an additional defensive layer.

F. Response–Tool Coordination

The principal coordination problem occurs when a response R_1 creates a tool call T_1, the user begins a new utterance U_2, and T_1 finishes after the redirection. VerbalVis tracks the conversational response and a monotonically increasing turn epoch so that the backend can decide whether a tool result still belongs to the active interaction.

The realtime session maintains the following coordination variables:

- `current_response_id`, identifying the active model response;
- `_turn_epoch`, incremented when user speech invalidates the current response;
- `_invalidated_response_ids`, containing superseded response identifiers;
- `_tool_tasks`, containing the asynchronous wrappers for active tool work;
- `_pending_tool_calls`, tracking unfinished calls for each response; and
- `_tool_state_lock`, serializing mutations of the shared analytical state.

When Qwen completes the arguments of a function call, the backend creates an asynchronous tool task. The task captures the current response identifier and the current value

TABLE III
SCHEMA-GROUNDED ANALYTICAL TOOLS IN THE ACTIVE VERBALVIS IMPLEMENTATION.

Tool	Principal arguments	State and visual effect
`filter_data`	`field`, `operator`, `value`, `append`	Adds or replaces an active data constraint and refreshes the relevant dashboard views. A special all-fields operation can clear the filter set.
`remove_filter`	`field`	Removes the constraint associated with one field while retaining the remaining filters.
`highlight_visual`	`view_id`	Sets the currently highlighted view without changing the filtered data population.
`append_visual`	`chart_type`, `x`, `y`, `title`	Creates a validated visualization and appends it to the dynamic workspace using a generated identifier.
`set_low_score_threshold`	`threshold`	Changes the operational threshold, between 1 and 5, used by low-score aggregations and refreshes affected views.
`delete_visual`	`view_id`	Removes a dynamic view and clears the highlight when it refers to the deleted view.

of `_turn_epoch`. This captured pair functions as the call's ownership metadata even though the implementation does not store a separate explicit owner object.

Before execution, the backend checks whether the call is stale. It performs this check once before acquiring the state lock and again after acquiring it. The tool handler is then executed through `asyncio.to_thread()`, after which the backend performs a third stale check. Abstractly, a result is eligible to be relayed when

$$\begin{aligned}\text{valid}(T) = {} & (T.\mathit{epoch} = \mathit{currentEpoch}) \\ & \wedge (T.\mathit{responseId} \notin \mathit{invalidatedResponses}) \\ & \wedge \mathit{sessionRunning}.\end{aligned} \tag{2}$$

For a valid call, the backend sends the tool result to the browser, sends a refreshed `views_update` when required, and returns a function-call output containing the updated dashboard context to Qwen. The pending-call counter is then decremented. When the last tool for a response has been finalized, the backend permits the provider to continue with a spoken response that reflects the tool outputs.

On barge-in, the speech-start handler increments the epoch, adds the current response identifier to the invalidated set, invokes `cancel()` on the tracked asynchronous tool tasks, sends `response.cancel` to Qwen, and notifies the browser to stop playback. All calls that captured the previous epoch subsequently fail one of the stale checks. Calls that have not yet entered their handler can therefore be prevented from executing, and completed stale calls are not relayed to the frontend or returned to the model as valid function outputs.

The distinction between physical cancellation and logical invalidation is important. Calling `cancel()` can stop an asynchronous task that has not entered blocking work. However, once `execute_tool()` is running inside `asyncio.to_thread()`, cancelling the awaiting coroutine does not terminate the underlying operating-system thread. The tool may therefore finish its DuckDB query and mutate the shared in-memory state before the post-execution stale check runs.

Consequently, the third stale check is a *relay decision*, not a transactional commit boundary. It prevents the stale result, tool message, and function-call output from being sent onward, but it cannot roll back a mutation that has already occurred inside the thread. The current prototype has no snapshot, undo, or transactional rollback mechanism. We therefore claim that VerbalVis attempts to cancel obsolete work and suppresses stale results from being relayed, rather than claiming that all stale computation or mutation is physically prevented.

This limitation is most relevant when the user interrupts during the interval between the second stale check and completion of the thread-based handler. The epoch mechanism remains useful for queued work and result suppression, but a production implementation should compute tool effects on an isolated state copy and merge them only after validation, or should use transaction-aware handlers that support cancellation and rollback.

G. Logging and Instrumentation

VerbalVis records both human-readable and structured logs for debugging, interaction reconstruction, and user-study analysis. Each session produces six text logs covering realtime events, tool calls, dashboard updates, barge-in events, connection events, and the conversation. It also produces `conversation.jsonl`, `tool_calls.jsonl`, and `session_summary.jsonl`.

The conversation log stores timestamps, session identifiers, roles, and final utterance text. The structured tool log stores the tool name, normalized arguments, response and call identifiers, result status, cancellation status, tool duration, captured turn epoch, event timeline, interaction mode, and a snapshot of the dashboard context. These records allow an analyst to associate tool activity with a conversational episode and determine whether a call was rejected by a stale check.

The current backend logs are sufficient to reconstruct tool-start latency and stale-result handling. They also provide a backend-side approximation of when an aligned dashboard update was produced. However, the Qwen path does not log the exact time at which the browser closes its audio context, and it does not produce a provider-side conversation-truncation event. Interrupt-to-audio-stop latency and wasted assistant speech therefore cannot be measured reliably from the current logs.

Similarly, the frontend does not send a `frontend_render_completed` acknowledgement after Vega-Lite has finished rerendering. The logs can identify when a `views_update` was sent, but not the exact time at which the corresponding visualization became visible.

User-study analyses should consequently distinguish backend update latency from true end-to-end visual-rendering latency.

Finally, although individual tool records include the interaction mode, the prototype does not emit a single authoritative session-level condition event. For the controlled study, the experimenter records the configured condition when launching each session. Adding explicit session-condition, playback-stop, dashboard-version, and render-completion events would make the instrumentation more complete and reduce the need to align multiple log streams post hoc.

H. Implementation Summary

The implementation realizes the primary interaction loop through continuous audio streaming, server-side speech detection, schema-grounded analytical tools, a persistent visual workspace, and response–tool ownership represented by response identifiers and turn epochs. Its strongest coordination mechanism is the repeated stale-call check that suppresses outputs belonging to superseded responses.

At the same time, the current system remains a research prototype. Its thread-based tool execution cannot guarantee physical cancellation, stale mutations cannot be rolled back, dashboard updates are not versioned, and the frontend provides no independent stale-update check. These boundaries are important for interpreting the system and evaluation: VerbalVis demonstrates how full-duplex redirection can be coupled to visual-analytics tools and logical result invalidation, but it does not yet provide transactional execution semantics.

VI. User Study and Technical Benchmark

A. Design

The study is designed to test the paper's central claim rather than general usability: whether a full-duplex supersession pathway makes analytical intent revision in EDA faster, more natural, and more controllable. The primary causal comparison is **not** between a realtime speech model and a cascaded speech pipeline. Instead, we compare two conditions that share the same model, voice, prompt, tool schema, dashboard, backend, VAD configuration, and context-reinjection logic. The only intended difference is whether user speech during assistant output triggers immediate response cancellation, epoch invalidation, and stale-action discard.

We plan a within-subjects, counterbalanced study with N=12–16 participants. Participants must have basic data-analysis experience, but need not know the Olist domain in advance. Each participant completes two open-ended exploration tasks on the Olist dataset, one per primary condition; task–condition assignment and condition order are counterbalanced. Each task lasts 12–15 minutes after a short tutorial.

- **Condition A (VerbalVis-FullDuplex):** audio is handled by the `gpt-realtime-2` session; the user can speak during assistant output; barge-in cancels or truncates the current response, increments `turn_epoch`, invalidates stale tool work, and triggers replanning against reinjected dashboard context.
- **Condition B (VerbalVis-NoBarge):** the system uses the same `gpt-realtime-2` session, voice, prompt, VAD settings, dashboard, backend, tool schema, and context-injection logic. However, user speech during assistant output is queued until the current response completes; it does not trigger `response.cancel`, `conversation.item.truncate`, immediate epoch invalidation, or stale-tool discard. This preserves the realtime model family while removing the supersession policy.

Supplementary ecological baseline. We additionally define **VerbalVis-Cascade** as an ecological baseline rather than the main causal comparison. In this condition, user speech is transcribed by ASR, passed to a text LLM using the same dashboard tool schema and Olist backend, and synthesized back to speech with TTS. The user must wait for the system to finish before the next request is processed. This baseline represents the speech-wrapped, turn-based text-LLM VA regime implicit in systems such as LightVA and DashChat, but it necessarily changes model type, speech pipeline, ASR/TTS latency, audio quality, turn-taking policy, and response timing. We therefore report Cascade results as supplementary context, not as evidence that the barge-in mechanism alone caused any observed advantage.

B. Tasks

Tasks are written to invite open-ended EDA while naturally eliciting the three revision types in our framework. Participants are not instructed to interrupt. The tutorial only explains the interaction regime of each condition: in FullDuplex they may speak whenever they want to redirect the analysis; in NoBarge their speech during assistant output is accepted but processed after the current response finishes. If Cascade is run, participants are told that it is a turn-based speech interface whose next request is processed only after the current response completes.

- **Customer satisfaction task.** "Explore what factors may be associated with low customer review scores. Produce three findings and one follow-up recommendation."
- **Sales, logistics, and region task.** "Investigate unusual order or revenue patterns and whether they relate to delivery or regional differences. Produce three findings and one follow-up recommendation."

C. Measurements

Quantitative.

- *Intent revision count* — number of user-initiated direction changes per session, coded from session video, transcript, dashboard events, and the `barge_in.log` timeline.
- *Revision type distribution* — proportion of revisions coded as Goal Shift, Hypothesis Correction, or Scope Narrowing.
- *Exploration breadth* — number of distinct analytical dimensions touched (fields, views, filter expressions) during the session.
- *Insight count* — number of substantive findings reported in the post-task summary, coded by two raters with Cohen's κ reported.

- *Time-to-revision* — elapsed time between the start of the redirecting utterance and the first system action aligned with the new intent, measured from video and event logs.
- *Wasted output time* — time during which the system continues producing the old analytical direction after the participant has begun redirecting.
- *Queue delay* — in NoBarge, the interval between redirecting speech onset and the moment the queued request begins processing.
- *Time-to-first-audio (TTFA)* and *tool-call duration*, recorded automatically by the existing `_response_metrics` instrumentation in `realtime.py`; if Cascade is run, ASR, text-LLM, and TTS latencies are logged separately and reported as component-level ecological costs.

Qualitative.

- Think-aloud transcripts, coded thematically.
- Semi-structured post-task interviews (10–15 min).
- NASA-TLX (six standard sub-scales).
- Perceived Control scale (3 Likert-7 items, e.g., "I felt able to redirect the analysis whenever I wanted.").

D. Episode Coding

The unit of analysis is a *revision episode*. Two coders inspect synchronized transcripts, screen recordings, dashboard event logs, and audio timelines. For each candidate episode, coders determine (i) whether it is a valid analytical intent revision rather than a clarification, backchannel, or unrelated speech fragment; (ii) whether it is Goal Shift, Hypothesis Correction, or Scope Narrowing; (iii) whether the system successfully responded to the revised intent; and (iv) the time from the start of the redirecting utterance to the first aligned system action, such as a tool call, dashboard update, or explicit spoken acknowledgment. Disagreements are resolved through discussion after reporting inter-rater agreement.

E. Hypotheses

- **H1:** Intent revision count is higher under VerbalVis-FullDuplex than under VerbalVis-NoBarge.
- **H2:** VerbalVis-FullDuplex reduces time-to-revision, queue delay, and wasted output time relative to VerbalVis-NoBarge.
- **H3:** VerbalVis-FullDuplex increases exploration breadth and insight count relative to VerbalVis-NoBarge.
- **H4:** Perceived control is higher under VerbalVis-FullDuplex, while NASA-TLX is not significantly higher.
- **H5:** The advantage of VerbalVis-FullDuplex is largest for *Hypothesis Correction* and *Scope Narrowing*, because these revisions are most costly when users must wait through an explanation they already intend to reject or constrain.

F. Technical Benchmark

To separate mechanism-level behavior from participant variability, we additionally plan a 250-trial automated benchmark comparing VerbalVis-FullDuplex and VerbalVis-NoBarge. Scripted audio scenarios cover five interruption families: no interruption, early goal shift, late goal shift, hypothesis correction, and scope narrowing. Each scenario is repeated across multiple seeds and dashboard states. The benchmark reports speech-onset detection latency, cancel/truncation latency, stale-tool discard rate, queued-request delay, post-supersession tool correctness, dashboard-context consistency, and component-level latency. The user study tests whether supersession-enabled full-duplex interaction changes analytical behavior and perceived control; the technical benchmark tests whether the mechanism itself reliably cancels stale work, discards stale calls, and preserves dashboard-context consistency.

G. Results (to be reported)

Placeholder. Tables and figures will report per-condition means and within-subject differences for the primary FullDuplex–NoBarge comparison with Wilcoxon signed-rank tests and effect sizes. Cascade, if collected, will be reported in a separate supplementary table to contextualize ecological latency and user experience, not to support the causal barge-in claim. Qualitative themes will be illustrated with representative quotations. The instrumentation already in place yields TTFA, per-turn latency, tool-call duration, stale-call status, and a per-session dashboard context snapshot for every tool invocation, providing both quantitative and trace data.

H. Anticipated Discussion

We expect the analyses to show that (a) the three revision types in our framework appear in the data with distinguishable distributions across participants; (b) the FullDuplex advantage over NoBarge is largest on Hypothesis Correction and Scope Narrowing episodes, where delayed processing forces users to passively witness explanations they already intend to reject or constrain; and (c) Perceived Control correlates positively with intent revision count and negatively with wasted output time. We further expect the automated benchmark to show that the gain is not merely due to the realtime model family, but to the supersession pathway that cancels stale output and preserves dashboard-context consistency.

VII. Discussion

Limitations. The current prototype implements five of the fifteen taxonomy operations as schema-based tools; while these operations cover the running example and planned study tasks, claims about the taxonomy's completeness require broader deployment. VerbalVis treats all barge-in events with the same downstream processing and does not classify revision type at runtime; the three types in Section **??** are analytical categories, not separate system policies. The planned study size (N=12–16) is appropriate for an exploratory within-subjects system study but still limits statistical power for subgroup analysis. VerbalVis-NoBarge is a controlled ablation rather than a naturally deployed product: it preserves the realtime model family while disabling immediate supersession, which is useful for causal attribution but less ecologically familiar. Conversely, VerbalVis-Cascade is ecologically meaningful because it represents a speech-wrapped text-LLM VA regime, but

it changes model type, speech pipeline, latency decomposition, audio quality, and turn-taking policy; we therefore treat it only as supplementary context. Finally, the Olist dataset, while rich, is domain-specific; generalization to clinical, financial, or scientific datasets remains future work.

Implications. Treating barge-in as a structural supersession signal expands the conversational VA design space without requiring the system to infer the user's meaning from timing alone. The interruption event gives the controller permission to protect the analytical state from stale work; the redirecting utterance provides the content of the revised intent. Future systems may also use the *timing* of an interruption (early vs. late in a response) as a weak cue about whether the user rejects the direction, the evidence, or the conclusion, but such use should remain subordinate to the utterance content and dashboard context.

Generalization beyond voice. The framework is voice-motivated but not voice-bound. Any modality with a true interruption primitive — including streaming-LLM text chat with mid-response edits, or gestural systems that can preempt animations — can be analyzed through the same Intent Evolution lens. We see VerbalVis as a concrete instantiation of a broader design pattern.

Future work. Beyond completing the tool taxonomy and broadening data domains, we plan (i) type-aware interruption handling that differentiates Goal Shift, Hypothesis Correction, and Scope Narrowing at runtime, (ii) a longitudinal study examining whether users' intent-revision strategies stabilize over repeated sessions, and (iii) an extension to multi-analyst sessions where barge-ins from different participants might conflict or require negotiated replanning.

VIII. Conclusion

We have presented VerbalVis, a full-duplex conversational visual analytics system motivated by the observation that user barge-in can serve as a structural supersession signal during analytical work. The Intent Evolution framework identifies three recurring revision types — Goal Shift, Hypothesis Correction, and Scope Narrowing — and motivates a reusable supersession control protocol for conversational EDA. Built on `gpt-realtime-2`, response epochs, schema-based tool calling, and an in-memory DuckDB analytics layer with continuously reinjected dashboard context, VerbalVis demonstrates how full-duplex speech can move intent revision from a between-turn afterthought to a continuously available action. A case study on the Olist e-commerce dataset illustrates all three revision types in a single session; a planned within-subjects user study will make its primary comparison between VerbalVis-FullDuplex and VerbalVis-NoBarge, with VerbalVis-Cascade retained only as a supplementary ecological baseline; and a 250-trial technical benchmark will test the reliability of the supersession mechanism. Together, these evaluations separate the human effect of full-duplex redirection from the mechanism-level question of whether stale work can be canceled or discarded while dashboard context remains consistent.

References

[1] D. M. Russell, M. J. Stefik, P. Pirolli, and S. K. Card, "The cost structure of sensemaking," in *Proceedings of the INTERCHI '93 Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 1993, pp. 269–276.

[2] P. Pirolli and S. Card, "The sensemaking process and leverage points for analyst technology as identified through cognitive task analysis," in *Proceedings of the International Conference on Intelligence Analysis*, vol. 5. McLean, VA, USA, 2005, pp. 2–4.

[3] D. Sacha, A. Stoffel, F. Stoffel, B. C. Kwon, G. Ellis, and D. A. Keim, "Knowledge generation model for visual analytics," *IEEE Transactions on Visualization and Computer Graphics*, vol. 20, no. 12, pp. 1604–1613, 2014.

[4] L. Battle and J. Heer, "Characterizing exploratory visual analysis: A literature review and evaluation of analytic provenance in tableau," *Computer Graphics Forum*, vol. 38, no. 3, pp. 145–159, 2019.

[5] T. Gao, M. Dontcheva, E. Adar, Z. Liu, and K. G. Karahalios, "DataTone: Managing ambiguity in natural language interfaces for data visualization," in *Proceedings of the 28th Annual ACM Symposium on User Interface Software and Technology*. Association for Computing Machinery, 2015, pp. 489–500.

[6] V. Setlur, S. E. Battersby, M. Tory, R. Gossweiler, and A. X. Chang, "Eviza: A natural language interface for visual analysis," in *Proceedings of the 29th Annual Symposium on User Interface Software and Technology*. Association for Computing Machinery, 2016, pp. 365–377.

[7] A. Narechania, A. Srinivasan, and J. T. Stasko, "NL4DV: A toolkit for generating analytic specifications for data visualization from natural language queries," *IEEE Transactions on Visualization and Computer Graphics*, vol. 27, no. 2, pp. 369–379, 2021.

[8] V. Dibia, "LIDA: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics: System Demonstrations*. Association for Computational Linguistics, 2023, pp. 113–126.

[9] C. Wang, B. Lee, S. M. Drucker, D. Marshall, and J. Gao, "Data formulator 2: Iteratively creating rich visualizations with ai," 2024.

[10] Y. Zhao, J. Wang, L. Xiang, X. Zhang, Z. Guo, C. Turkay, Y. Zhang, and S. Chen, "LightVA: Lightweight visual analytics with LLM agent-based task planning and execution," *IEEE Transactions on Visualization and Computer Graphics*, 2024, early Access.

[11] G. Klein, J. K. Phillips, E. L. Rall, and D. A. Peluso, "A data–frame theory of sensemaking," in *Expertise Out of Context: Proceedings of the Sixth International Conference on Naturalistic Decision Making*, R. R. Hoffman, Ed. New York: Lawrence Erlbaum Associates, 2007, pp. 113–155.

[12] M. J. Bates, "The design of browsing and berrypicking techniques for the online search interface," *Online Review*, vol. 13, no. 5, pp. 407–424, 1989.

[13] A. Srinivasan and J. T. Stasko, "Orko: Facilitating multimodal interaction for visual exploration and analysis of networks," *IEEE Transactions on Visualization and Computer Graphics*, vol. 24, no. 1, pp. 511–521, 2018.

[14] A. Srinivasan, B. Lee, N. H. Riche, S. M. Drucker, and K. Hinckley, "InChorus: Designing consistent multimodal interactions for data visualization on tablet devices," in *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 2020, pp. 1–13.

[15] A. Srinivasan, B. Lee, and J. T. Stasko, "Interweaving multimodal interaction with flexible unit visualizations for data exploration," *IEEE Transactions on Visualization and Computer Graphics*, vol. 27, no. 8, pp. 3519–3533, 2021.

[16] A. Srinivasan and V. Setlur, "Snowy: Recommending utterances for conversational visual analysis," in *Proceedings of the 34th Annual ACM Symposium on User Interface Software and Technology*. Association for Computing Machinery, 2021, pp. 864–880.

[17] R. Mitra, A. Narechania, A. Endert, and J. T. Stasko, "Facilitating conversational interaction in natural language interfaces for visualization," in *2022 IEEE Visualization and Visual Analytics Conference*. IEEE, 2022, pp. 6–10.

[18] C. Wang, J. Thompson, and B. Lee, "Data formulator: Ai-powered concept-driven visualization authoring," *IEEE Transactions on Visualization and Computer Graphics*, vol. 30, no. 1, pp. 1128–1138, 2024.

[19] P. Ma, R. Ding, S. Wang, S. Han, and D. Zhang, "InsightPilot: An LLM-empowered automated data exploration system," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 2023, pp. 346–352.

[20] L. Xie, C. Zheng, H. Xia, H. Qu, and Z.-T. Chen, "WaitGPT: Monitoring and steering conversational LLM agent in data analysis with on-the-fly code visualization," in *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*. Association for Computing Machinery, 2024.

[21] L. Weng, X. Wang, J. Lu, Y. Feng, Y. Liu, H. Feng, D. Huang, and W. Chen, "InsightLens: Augmenting llm-powered data analysis with interactive insight management and navigation," *IEEE Transactions on Visualization and Computer Graphics*, vol. 31, no. 6, pp. 3719–3732, 2025.

[22] G. Skantze, "Turn-taking in conversational systems and human-robot interaction: A review," *Computer Speech & Language*, vol. 67, p. 101178, 2021.

[23] R. Heins, M. Franzke, and A. Bayya, "Turn-taking as a design principle for barge-in in spoken language systems," *International Journal of Speech Technology*, vol. 2, pp. 155–164, 1997.

[24] T.-E. Lin, Y. Wu, F. Huang, L. Si, J. Sun, and Y. Li, "Duplex conversation: Towards human-like interaction in spoken dialogue systems," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, 2022, pp. 3299–3308.

[25] B. Veluri, B. N. Peloquin, B. Yu, H. Gong, and S. Gollakota, "Beyond turn-based interfaces: Synchronous LLMs as full-duplex dialogue agents," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024, pp. 21 390–21 402.

[26] A. Défossez, L. Mazaré, M. Orsini, A. Royer, P. Pérez, H. Jégou, E. Grave, and N. Zeghidour, "Moshi: A speech-text foundation model for real-time dialogue," 2024.

[27] W. Yu, S. Wang, X. Yang, X. Chen, X. Tian, J. Zhang, G. Sun, L. Lu, Y. Wang, and C. Zhang, "SALMONN-omni: A standalone speech LLM without codec injection for full-duplex conversation," in *Advances in Neural Information Processing Systems*, 2025.

[28] P. Vakkari, "Changes in search tactics and relevance judgements when preparing a research proposal: A summary of the findings of a longitudinal study," *Information Retrieval*, vol. 4, no. 3–4, pp. 295–310, 2001.

[29] D. Klahr and K. Dunbar, "Dual space search during scientific reasoning," *Cognitive Science*, vol. 12, no. 1, pp. 1–48, 1988.

[30] C. A. Chinn and W. F. Brewer, "The role of anomalous data in knowledge acquisition: A theoretical framework and implications for science instruction," *Review of Educational Research*, vol. 63, no. 1, pp. 1–49, 1993.

[31] P. Pirolli and S. K. Card, "Information foraging," *Psychological Review*, vol. 106, no. 4, pp. 643–675, 1999.

[32] M. Brehmer and T. Munzner, "A multi-level typology of abstract visualization tasks," *IEEE Transactions on Visualization and Computer Graphics*, vol. 19, no. 12, pp. 2376–2385, 2013.

[33] K. Wongsuphasawat, D. Moritz, A. Anand, J. Mackinlay, B. Howe, and J. Heer, "Voyager: Exploratory analysis via faceted browsing of visualization recommendations," *IEEE Transactions on Visualization and Computer Graphics*, vol. 22, no. 1, pp. 649–658, 2016.

[34] S. Pontis and A. Blandford, "Understanding "influence": An empirical test of the data–frame theory of sensemaking," *Journal of the Association for Information Science and Technology*, vol. 67, no. 4, pp. 841–858, 2016.